Gestión de la Configuración del Software

Grupo 5



¿Qué es la Gestión de la Configuración?

Es la disciplina que identifica los elementos que componen el software, controla sus cambios y deja registro de cada versión.

1

Orden

Sabemos exactamente qué archivos forman el sistema.

2

Control

Ningún cambio entra sin quedar registrado.

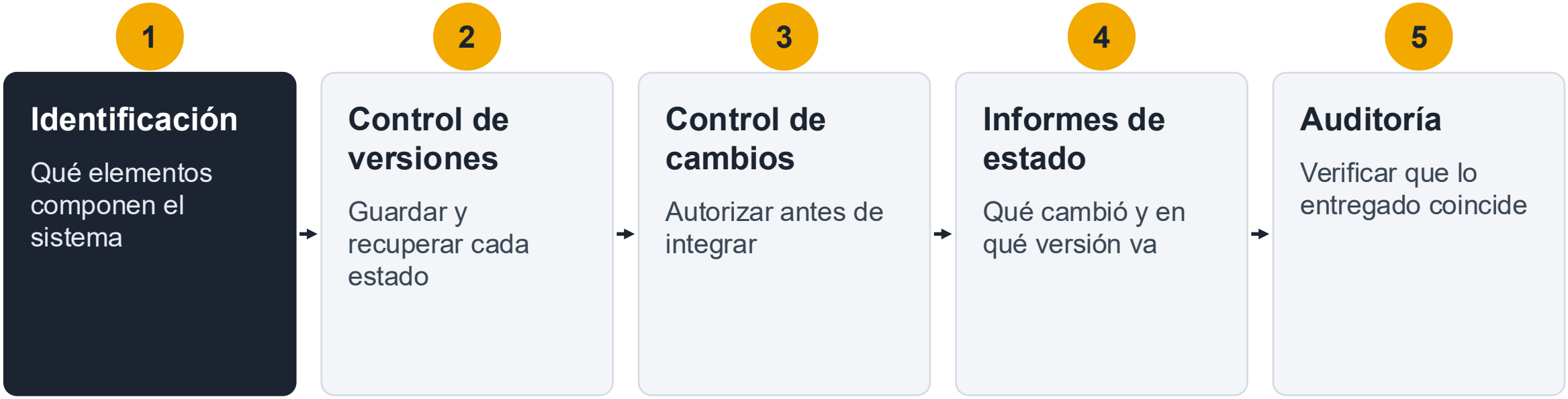
3

Trazabilidad

Quién cambió qué, cuándo y por qué.

Sin esto no se sabe qué se entregó ni cómo volver atrás.

Las cinco actividades



Sin identificar no hay qué versionar, y sin registro no hay qué auditar.

Identificación de la configuración

Es decidir qué elementos quedan bajo control y darle a cada uno nombre y versión.

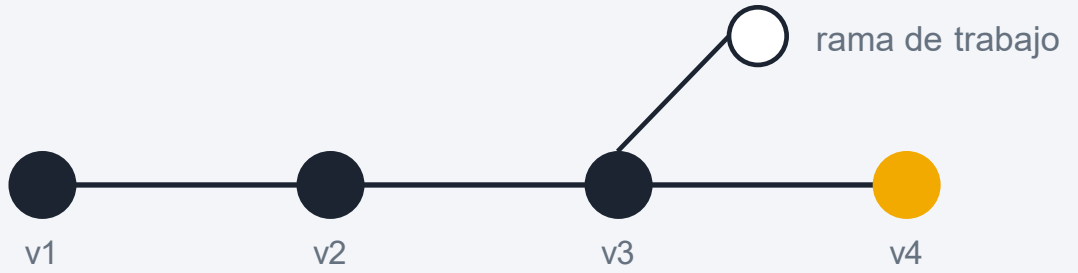
Código fuente	**Archivos de datos**	**Archivos compilados**	**Scripts**	**Requerimientos**
Diseño	**Arquitectura**	**Planes de prueba**	**Procedimientos de prueba**	**Planes de calidad**

Un elemento que no está identificado no se puede controlar, ni versionar, ni recuperar.

El seguimiento se hace durante todo el ciclo de vida: cada elemento se registra cuando nace, cada vez que cambia y cuando se entrega.

Control de versiones

Es el registro histórico de cada cambio, que permite volver a cualquier estado anterior.



Cada punto es un estado guardado al que se puede volver.

Sin control de versiones el equipo guarda copias sueltas y nadie sabe cuál es la buena.

Para qué sirve

Recuperar un estado anterior cuando algo se rompe.

En equipo

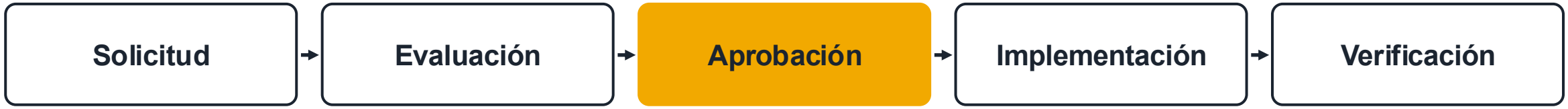
Varios trabajan a la vez sin pisarse el trabajo.

Herramientas

Git es el sistema; GitHub, GitLab y Bitbucket lo alojan.

Control de cambios

Es el procedimiento para solicitar, evaluar, aprobar e integrar una modificación.



Sin control

— Nadie sabe por qué cambió una línea.
— No se puede revertir con seguridad.
— Se rompe lo que ya estaba probado.
— Dos personas editan el mismo archivo.

Con control

— Cada cambio queda documentado.
— Alguien más lo revisa antes de entrar.
— Se puede volver a la versión anterior.
— El sistema evoluciona sin perder historia.

Pruebas relacionadas con la configuración

	Caja negra	Caja gris	Caja blanca
Qué conoce	Nada del interior	Diseño y arquitectura	El código completo
Qué prueba	Entradas y salidas	Flujos e integración	Rutas del código
Necesita el fuente	No	En parte	Sí

Funcionales

Que haga lo que pide el requisito

Rendimiento

Que responda en tiempo aceptable

Defectos de código

Errores de lógica y malas prácticas

Seguridad

Accesos, claves y permisos

Si no existe el código fuente, la caja blanca no es posible: solo queda probar desde fuera.

Nuestro caso: el sistema heredado

Recibimos RecursosHumanos: una aplicación VB.NET compilada para .NET Framework 2.0, entregada solo en binarios.

108 archivos recibidos

0 de código fuente

3 versiones de la misma librería

1 base de datos sin respaldo

XML RecursosHumanos.exe.config ×

Insumos base › Sistema

```
3    <connectionStrings>
4      <add name="CMIConnectionString"
5        connectionString=
6          "Data Source=(localdb)\MSSQLLocalDB;
7           Initial Catalog=CMI;
8           User ID=user_1;
9           Password=**********" />
10   </connectionStrings>
```

La contraseña está tapada en esta lámina; en el repositorio está en texto plano.

Sin código fuente

No se puede recompilar ni auditar por dentro.

Credenciales versionadas

Usuario y contraseña viajan en el repositorio.

Falta el respaldo de la base

El script lo exige y no está en el proyecto.

Elementos que pondremos bajo control

Tipo de elemento	En nuestro sistema	Estado
Archivos compilados	RecursosHumanos.exe, GRLL.dll, ClassRRHH.dll	**Versionado por entrega**
Librerías de terceros	DevExpress v7.3, v9.1 y v11.2	**Fijar una sola versión**
Archivos de datos	Base CMI y su respaldo .bak	**Falta el respaldo**
Scripts	setup.sql	**Versionado**
Configuración	RecursosHumanos.exe.config	**Sacar credenciales**
Documentación	Requerimientos, diseño y arquitectura	**Por elaborar**
Planes de prueba	Casos de caja negra sobre el ejecutable	**Por elaborar**

Cada elemento tendrá un responsable y una versión identificada antes de la siguiente entrega.

Nuestro plan de gestión

1 Repositorio

- Git como herramienta.
- Alojado en GitHub, repositorio privado.
- Carpetas: sistema, scripts, docs y pruebas.

2 Qué se versiona

- Sí: scripts, documentos y configuración.
- No: credenciales ni respaldos pesados.
- Los compilados, solo como entregable.

3 Reglas del equipo

- Una rama por tarea.
- Mensaje con prefijo: Fix, Mejora o Doc.
- Un compañero revisa antes de integrar.
- Una etiqueta por entrega semanal.

4 Pruebas iniciales

- Caja negra: login y listados.
- Funcionales de los módulos principales.
- Seguridad: sacar las credenciales.

Conclusiones

1. Lo que no está identificado no se puede controlar ni recuperar.
2. Versionar y controlar cambios da trazabilidad: quién, qué y por qué.
3. Sin código fuente, la configuración y la caja negra son nuestra única defensa.

Gracias

¿Preguntas?

